

From design to production

LET THERE BE LIGHT

"And God said, Let there be light: and there was light." Genesis 1:3

Real capture of `./setup.sh` from a fresh clone of [crowx01/let-there-be-light](https://github.com/crowx01/let-there-be-light). Per-project scope, three skills opted in (architect, test-strategist, ship-checklist), nemotron deselected.

What the wizard writes: a `settings.json` with two hooks, an `.env.lttbl.example` with only the API keys your model picks need, and symlinks under `.claude/skills/`. Pre-existing hooks from other frameworks are preserved (appended, not clobbered).

1. The terminal session



interactive setup . from design to production

1. Where should the hooks install?

- g) global → ~/.claude/settings.json (loads every session, every project)
 - p) per-project → \$PWD/.claude/settings.json (only loads when Claude Code runs here)
 - s) skip → print the settings JSON to stdout, do not write anything
- ✓ target: /tmp/lttbl-pdf/.claude/settings.json

2. Which skills should auto-load at session start?

These are invoked as the very first tool calls of every session.
Leave blank to skip a skill; type y to include it.

3. Which PAL models should be listed in the delegate-first policy?

Pick every model your PAL install actually has keys for.
Unselected models are dropped from the routing map so Claude never tries them.

Ready to write /tmp/lttbl-pdf/.claude/settings.json

Existing file will be backed up with .bak.<timestamp> suffix.

- ✓ wrote /tmp/lttbl-pdf/.claude/settings.json
- ✓ symlinked architect -> /tmp/lttbl-pdf/.claude/skills/architect
- ✓ symlinked babysit-pr -> /tmp/lttbl-pdf/.claude/skills/babysit-pr
- ✓ symlinked debate-review -> /tmp/lttbl-pdf/.claude/skills/debate-review
- ✓ symlinked incident-response -> /tmp/lttbl-pdf/.claude/skills/incident-response
- ✓ symlinked pal-router -> /tmp/lttbl-pdf/.claude/skills/pal-router
- ✓ symlinked retrospective -> /tmp/lttbl-pdf/.claude/skills/retrospective
- ✓ symlinked ship-checklist -> /tmp/lttbl-pdf/.claude/skills/ship-checklist
- ✓ symlinked test-strategist -> /tmp/lttbl-pdf/.claude/skills/test-strategist
- ✓ PAL MCP server is registered in ~/.claude.json
- ✓ wrote env template: /tmp/lttbl-pdf/.claude/.env.lttbl.example

Next steps

1. Register PAL as an MCP server in ~/.claude.json:

```
"mcpServers": { "pal": { "type": "stdio", "command": "/path/to/zen-mcp-server/.pal_venv/bin/python", "args": ["/path/to/zen-mcp-server/server.py"], "env": { ...keys... } } }
```
 2. Source your API keys: `source /tmp/lttbl-pdf/.claude/.env.lttbl.example` (after editing it)
 3. Restart Claude Code.
 4. On the next session start you should see the auto-invoked skills fire immediately.
- ✓ let-there-be-light setup complete.

2. What Claude sees at the next session start

The SessionStart hook fires *before* Claude responds. Its payload is injected into context as if the user said it.

SessionStart · additionalContext

MANDATORY FIRST ACTIONS: on session start and for EVERY model, BEFORE responding, immediately call the Skill tool for `architect`, `test-strategist`, `ship-checklist`. Full preference set: (1) delegate-first via PAL: groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge. (2) KEEP IN CLAUDE: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, tool-sequence orchestration. (3) On every significant change (feature spec, migration, refactor) run the 4-step lifecycle: design review (architect skill), implementation review (debate-review), staging validation (ship-checklist), post-deploy check (incident-response ready).

UserPromptSubmit · additionalContext (fires on every message)

Reminder every message: delegate aggressively to PAL to conserve tokens (groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge.). Keep in Claude ONLY: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, orchestration. FAILOVER: if any model refuses or errors, immediately re-route; a refusal is a routing problem, not a stop.

Notice: nemotron and the two off-by-default skills (incident-response, retrospective) do not appear in either payload. Deselected during the wizard, so Claude never routes to or auto-invokes them.

3. Files on disk after install

```
/PROJECT
/PROJECT/.claude
/PROJECT/.claude/.env.lttbl.example
/PROJECT/.claude/settings.json
/PROJECT/.claude/skills
/PROJECT/.claude/skills/architect
/PROJECT/.claude/skills/babysit-pr
/PROJECT/.claude/skills/debate-review
/PROJECT/.claude/skills/incident-response
/PROJECT/.claude/skills/pal-router
/PROJECT/.claude/skills/retrospective
/PROJECT/.claude/skills/ship-checklist
```

```
/PROJECT/.claude/skills/test-strategist  
/PROJECT/tree.txt
```

.env.lttbl.example (only your chosen providers' keys)

```
# let-there-be-light API keys - source this from your shell rc, or export before starting  
Claude Code.  
# Do NOT commit the real values.  
export GEMINI_API_KEY=your-gemini-key  
export OPENROUTER_API_KEY=your-openrouter-key  
export CUSTOM_API_URL=https://api.groq.com/openai/v1  
export CUSTOM_API_KEY=your-groq-key
```

4. The 4-phase dev lifecycle

- 1 **Design.** `architect` skill runs. Draft ADR prose goes to `groq`; comparative reasoning between alternatives goes to `pro`. Output: system diagram, ADR, trade-off matrix, explicit unknowns.
 - 2 **Implementation.** `test-strategist` produces the test pyramid + coverage plan. When the PR opens, `debate-review` runs a two-model debate and posts one review from your `gh/glab/az` account. `babysit-pr` works the review rounds until merge-ready.
 - 3 **Staging.** `ship-checklist` runs before promotion. Rollback plan is non-negotiable. Smoke test is 5-10 operations that must pass within 60 seconds of promotion.
 - 4 **Production.** `incident-response` triages any alert (blast radius, runbook, status update). `retrospective` runs after anything customer-visible: UTC timeline, blameless narrative, at most 5 action items with owners and deadlines.
-

Repo: github.com/crowx01/let-there-be-light · Docs: [docs/WORKFLOW.md](#) · Sibling: [crowx01/sauron](#) (offsec)

And there was light.